



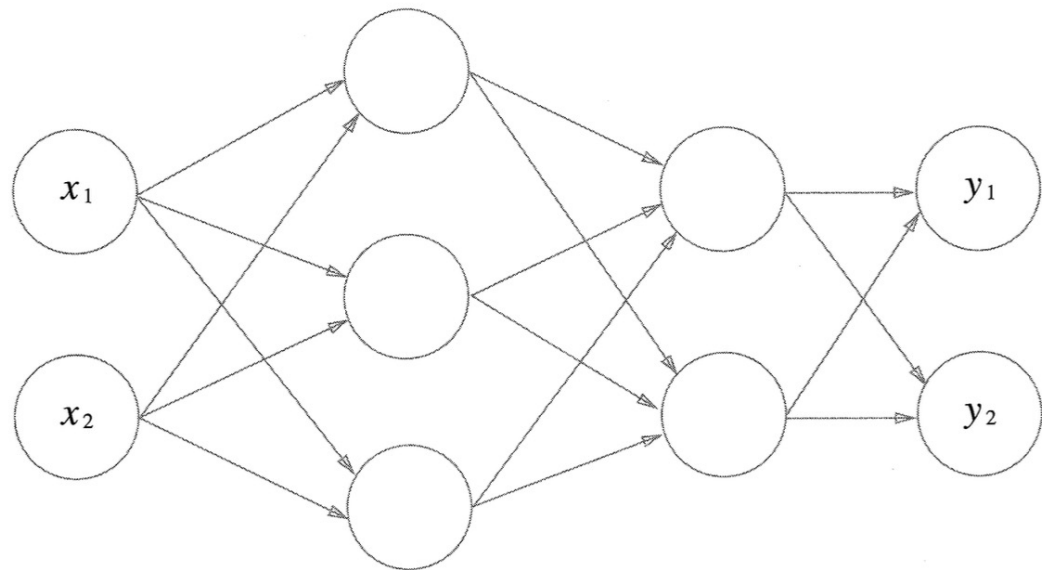
지능로봇 기계학습

2026 Spring

- 3층 작성

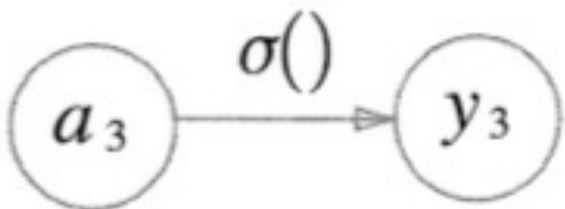
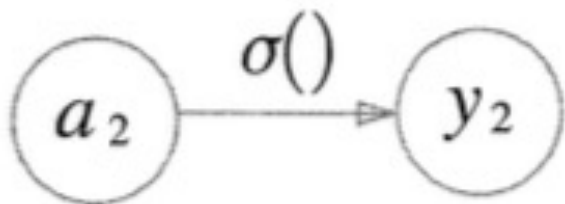
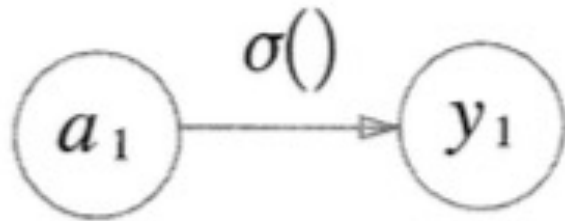
(a) activation function 을 sigmoid 함수로

(b) activation function을 relu 함수로



출력층 설계 (항등함수)

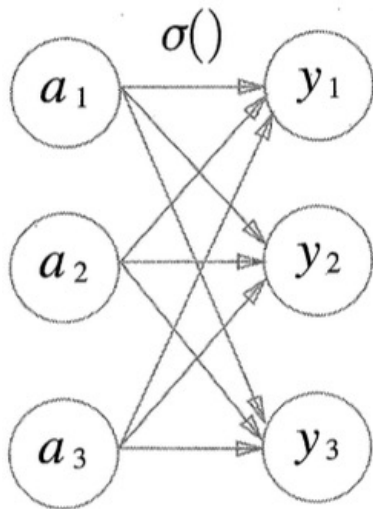
- 입력신호 $\rightarrow$ 출력신호



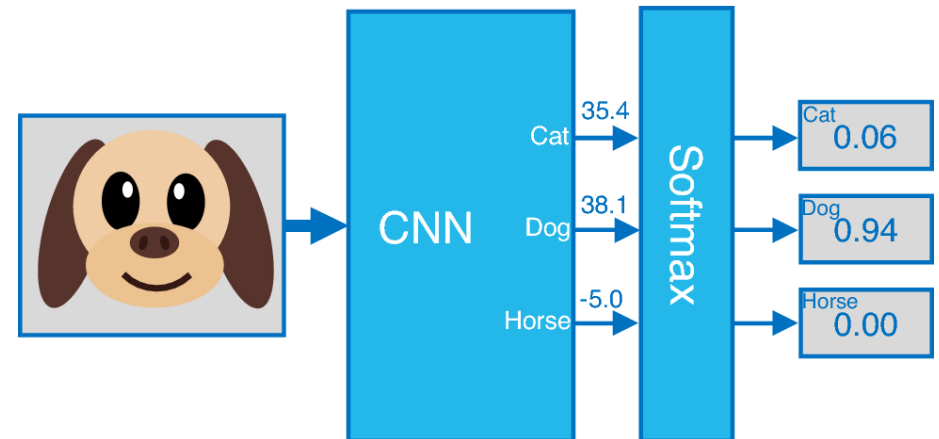
출력층 설계 – softmax function

$$y_k = \frac{\exp(a_k)}{\sum_{i=1}^n \exp(a_i)}$$

- $\exp(x)$: e^x 지수함수
- n : 출력층의 뉴런수
- y_k : k 번째 출력



- 분자 : 입력신호의 지수함수
- 분모 : 모든 입력신호의 지수함수의 합

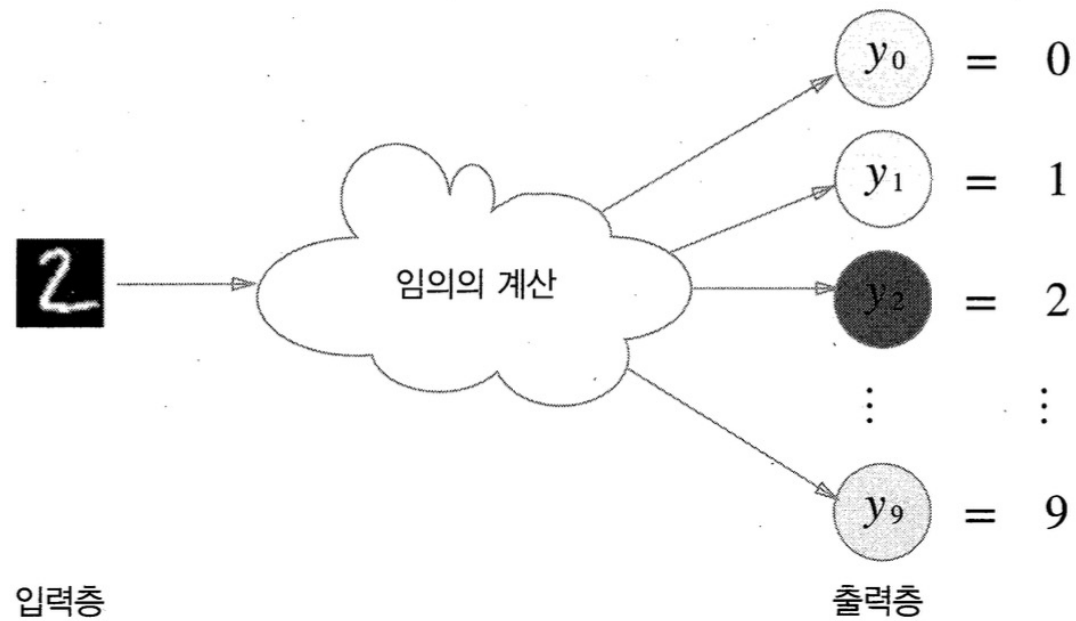


Softmax 구현시 주의사항

- e^{10} : over 20000, e^{100} : over 10^{40} , e^{1000} : inf
→ 큰 값끼리 나눗셈하면 결과 수치 불안정

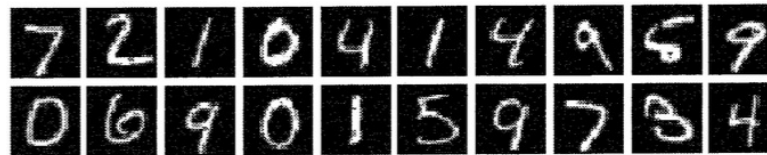
$$\begin{aligned} y_k &= \frac{\exp(a_k)}{\sum_{i=1}^n \exp(a_i)} = \frac{C \exp(a_k)}{C \sum_{i=1}^n \exp(a_i)} \\ &= \frac{\exp(a_k + \log C)}{\sum_{i=1}^n \exp(a_i + \log C)} \\ &= \frac{\exp(a_k + C')}{\sum_{i=1}^n \exp(a_i + C')} \end{aligned}$$

출력층의 뉴런 수 정하기



Mnist (이미지 데이터셋)

- 28*28 회색조 이미지
- 각 픽셀은 0~255값
- 각 이미지에는 각 숫자 레이블



```
1 import sys, os
2 sys.path.append(os.pardir)
3 from dataset.mnist import load_mnist
4
5 (x_train, t_train), (x_test, t_test) = load_mnist(flatten=True, normalize=False)
6
7 print(x_train.shape)
8 print(t_train.shape)
9 print(x_test.shape)
10 print(t_test.shape)
```

```
limsc@limsc-dguui-Mac-Studio Codes % /usr/bin/python3 "/Volun
Converting train-images-idx3-ubyte.gz to NumPy Array ...
Done
Converting train-labels-idx1-ubyte.gz to NumPy Array ...
Done
Converting t10k-images-idx3-ubyte.gz to NumPy Array ...
Done
Converting t10k-labels-idx1-ubyte.gz to NumPy Array ...
Done
Creating pickle file ...
Done!
(60000, 784)
(60000,)
(10000, 784)
(10000,)
```



```
def load_mnist(normalize=True, flatten=True, one_hot_label=False):
    """MNIST 데이터셋 읽기

    Parameters
    -----
    normalize : 이미지의 픽셀 값을 0.0~1.0 사이의 값으로 정규화할지 정한다.
    one_hot_label :
        one_hot_label이 True면, 레이블을 원-핫(one-hot) 배열로 돌려준다.
        one-hot 배열은 예를 들어 [0,0,1,0,0,0,0,0,0,0]처럼 한 원소만 1인 배열이다.
    flatten : 입력 이미지를 1차원 배열로 만들지를 정한다.
```

Returns

(훈련 이미지, 훈련 레이블), (시험 이미지, 시험 레이블)

"""

```
if not os.path.exists(save_file):
    init_mnist()
```

```
with open(save_file, 'rb') as f:
    dataset = pickle.load(f)
```

```
if normalize:
    for key in ('train_img', 'test_img'):
        dataset[key] = dataset[key].astype(np.float32)
        dataset[key] /= 255.0
```

True : 0~1, False : 0~255(유지)

```
if one_hot_label:
    dataset['train_label'] = _change_one_hot_label(dataset['train_label'])
    dataset['test_label'] = _change_one_hot_label(dataset['test_label'])
```

True : [0,1,0,0,0,0,0,0,0,0]

False : 숫자형태 레이블

```
if not flatten:
    for key in ('train_img', 'test_img'):
        dataset[key] = dataset[key].reshape(-1, 1, 28, 28)
```

True : 1차원

False : 1*28*28

```
return (dataset['train_img'], dataset['train_label']), (dataset['test_img'], dataset['test_label'])
```

```

1
2 import sys, os
3 sys.path.append(os.pardir) # 부모 디렉터리의 파일을 가져올 수 있도록 설정
4
5 import numpy as np
6 from dataset.mnist import load_mnist
7 from PIL import Image
8
9 def img_show(img):
10     pil_img = Image.fromarray(np.uint8(img))
11     pil_img.show()
12
13 (x_train, t_train), (x_test, t_test) = load_mnist(flatten=True, normalize=False)
14
15 img = x_train[0]
16 label = t_train[0]
17 print(label) # 5
18
19 print(img.shape) # (784,)
20 img = img.reshape(28, 28) # 형상을 원래 이미지의 크기로 변형
21 print(img.shape) # (28, 28)
22
23 img_show(img)
24

```

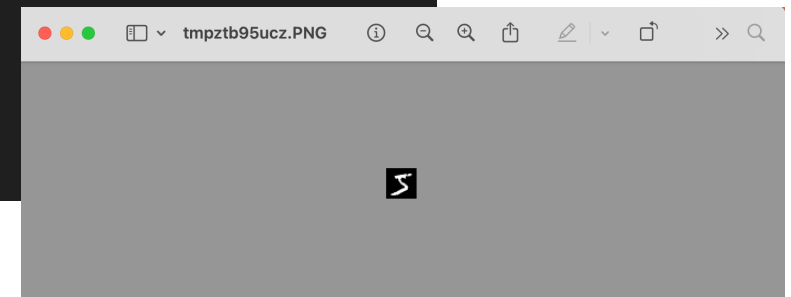
Python image library

Flatten : true

```

● limsc@Limsc-dguu
5
(784,)
(28, 28)

```



```
def get_data():
    (x_train, t_train), (x_test, t_test) = load_mnist(normalize=True, flatten=True, one_hot_label=False)
    return x_test, t_test
```

```
def init_network():
    with open(os.path.dirname(__file__) + "/3_week_sample_weight_minist.pkl", 'rb') as f:
        network = pickle.load(f)
    return network
```

Pickle 파일에 저장된 값을 읽음.

```
def predict(network, x):
    W1, W2, W3 = network['W1'], network['W2'], network['W3']
    b1, b2, b3 = network['b1'], network['b2'], network['b3']

    a1 = np.dot(x, W1) + b1
    z1 = sigmoid(a1)
    a2 = np.dot(z1, W2) + b2
    z2 = sigmoid(a2)
    a3 = np.dot(z2, W3) + b3
    y = softmax(a3)
    return y
```

```
def sigmoid(x):
    return 1 / (1 + np.exp(-x))
```

```
def softmax(a):
    c = np.max(a)
    exp_a = np.exp(a-c)
    sum_exp_a = np.sum(exp_a)
    y = exp_a / sum_exp_a
    return y
```

```
43 x, t = get_data()
44 print(x.shape)
45 network = init_network()
46 accuracy_cnt = 0
47 for i in range(len(x)):
48     y = predict(network, x[i])
49     p = np.argmax(y) # 확률이 가장 높은 원소의 인덱스를 얻는다.
50     if p == t[i]:
51         accuracy_cnt += 1
52
53 print("Accuracy:" + str(float(accuracy_cnt) / len(x)))
```

● limsc@Limsc-dguui-Mac-
(10000, 784)
Accuracy:0.9352

정규화(Normalization)

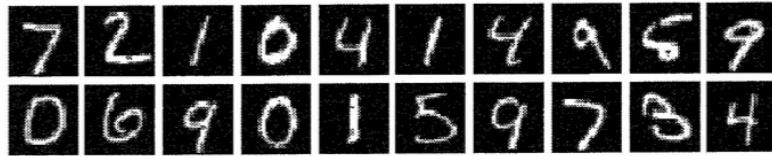
- 데이터의 특정 범위로 변환하는 처리

0~255 → 0.0~1.0

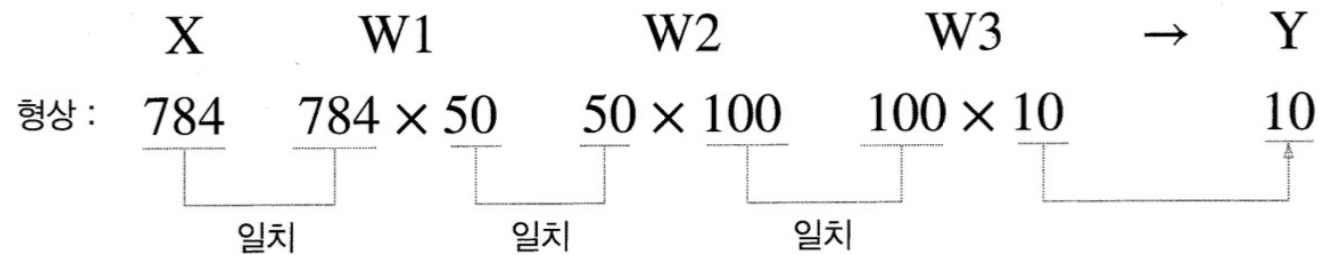
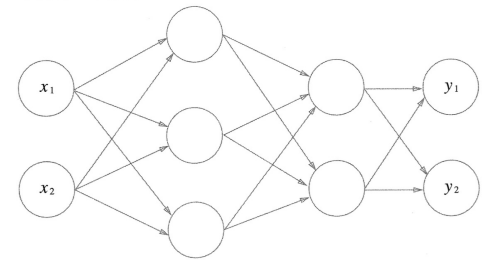
전처리(pre-processing)

- 신경망 입력데이터에 특정 변환을 가하는 것

배치처리

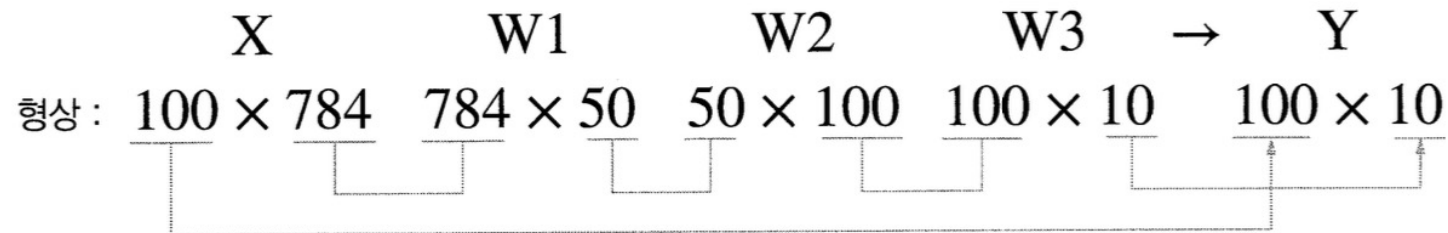
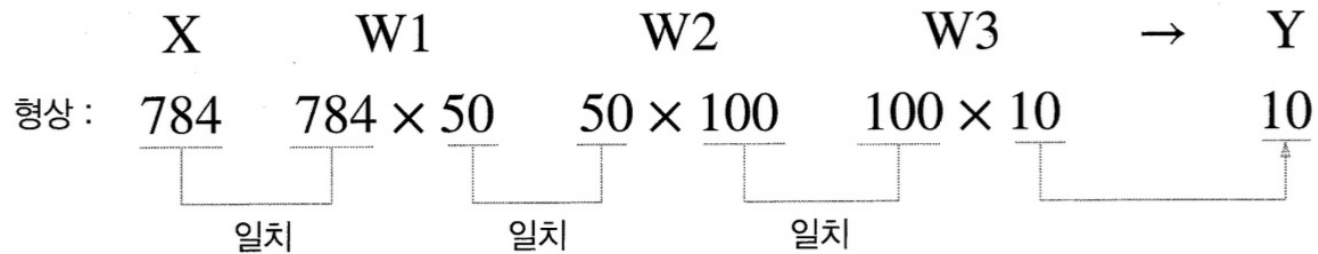


```
>>> x, _ = get_data()
>>> network = init_network()
>>> W1, W2, W3 = network['W1'], network['W2'], network['W3']
>>>
>>> x.shape
(10000, 784)
>>> x[0].shape
(784,)
>>> W1.shape
(784, 50)
>>> W2.shape
(50, 100)
>>> W3.shape
(100, 10)
```



배치처리

- 입력 784, 출력 10 → 입력 100*784, 출력 100*10



```

43 x, t = get_data()
44 print(x.shape)
45 network = init_network()
46 accuracy_cnt = 0
47 for i in range(len(x)):
48     y = predict(network, x[i])
49     p = np.argmax(y) # 확률이 가장 높은 원소의 인덱스를 얻는다.
50     if p == t[i]:
51         accuracy_cnt += 1
52
53 print("Accuracy:" + str(float(accuracy_cnt) / len(x)))

```



```

44 x, t = get_data()
45 network = init_network()
46
47 batch_size = 100 # 배치 크기
48 accuracy_cnt = 0
49
50     0~len(x)까지 batch_size의 간격으로 이루어진 리스트 반환
51 for i in range(0, len(x), batch_size):
52     x_batch = x[i:i+batch_size]
53     y_batch = predict(network, x_batch)
54     p = np.argmax(y_batch, axis=1)
55     accuracy_cnt += np.sum(p == t[i:i+batch_size])
56
57 print("Accuracy:" + str(float(accuracy_cnt) / len(x)))

```

```

>>> list( range(0, 10) )
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
>>> list( range(0, 10, 3) )
[0, 3, 6, 9]

```

```

>>> y = np.array([1, 2, 1, 0])
>>> t = np.array([1, 2, 0, 0])
>>> print(y==t)
[True True False True]
>>> np.sum(y==t)
3

```

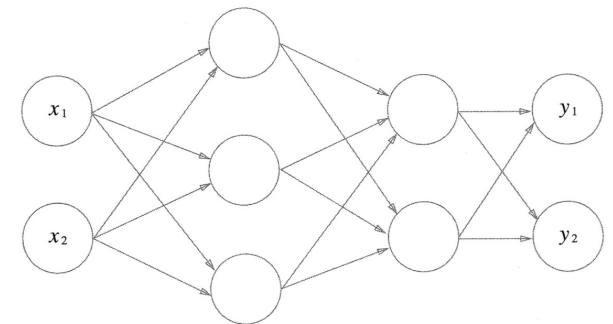


지능로봇 기계학습 - 신경망학습

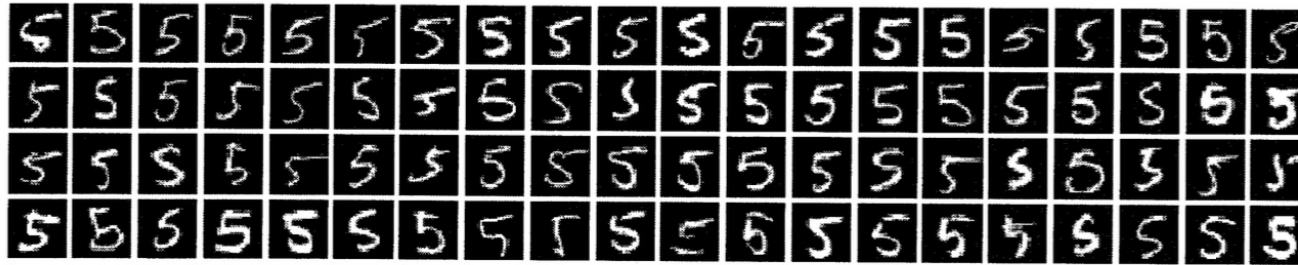
2025 Spring

학습?

훈련 데이터로부터
가중치 매개변수의 최적값을
자동으로 획득하는 것



데이터에서 학습!



- 이미지에서 특징(Feature) 추출
- 특징의 패턴을 기계학습 기술로 학습

End-to-End Machine Learning

- 데이터(입력) 에서 결과(출력)를 사람의 개입없이 얻음.

훈련데이터/시험데이터

- 훈련 데이터(Training Data)

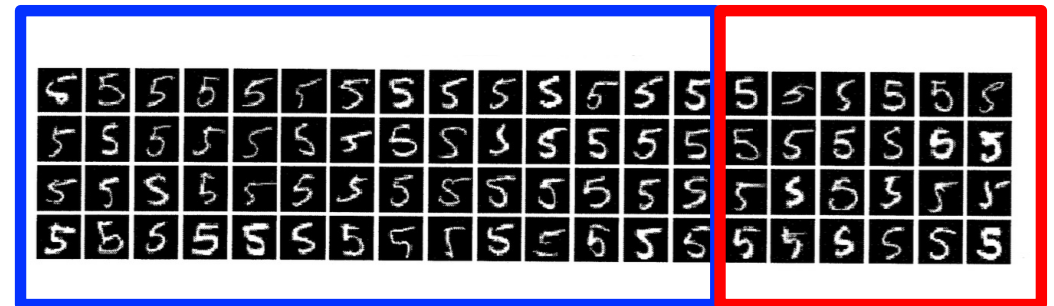
- : 학습에 사용하는 데이터

- : 훈련데이터를 사용해 최적의 매개변수를 찾음

훈련 데이터 시험 데이터

- 시험 데이터(Test Data)

- : 훈련한 모델의 성능을 평가하는데 활용



- 왜 나눌까?

- : 범용성을 확보하기 위해서 학습에 활용하지 않은 데이터라도 원하는 값을 만들어내기 위해

손실함수 (Loss Function)

Supervised learning : Classification (binary)

- Pass/non-pass based on time spent

x (hours)	y (pass/fail)
10	P
9	P
3	F
2	F

Supervised Learning : Classification(Multi-label)

- Letter grade (A, B, ...) based on time spent

x (hours)	y (grade)
10	A
9	B
3	D
2	F

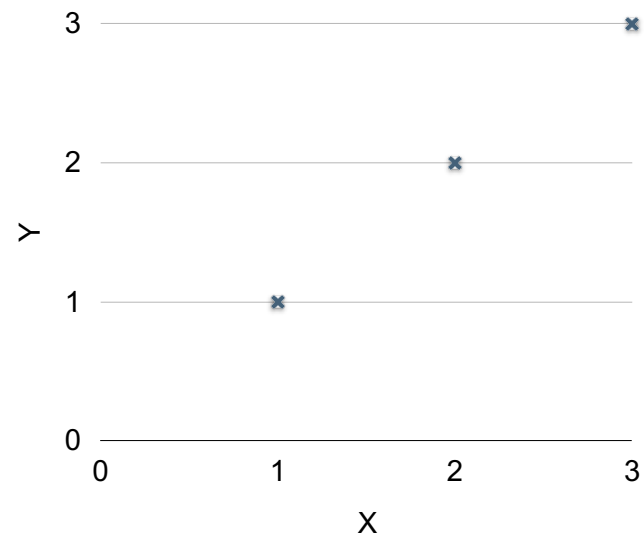
Supervised Learning : Regression

- Predicting final exam score based on time spent

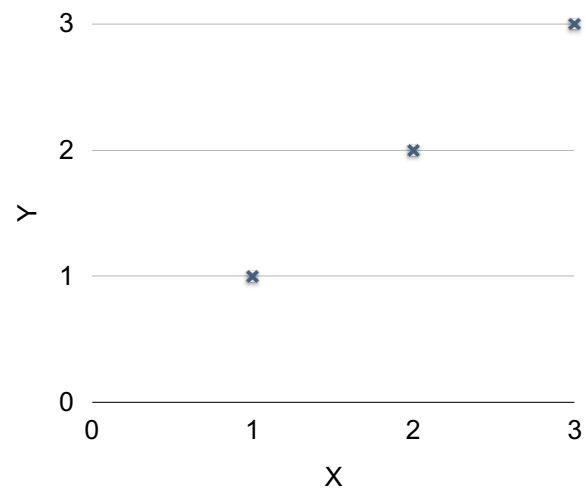
x (hours)	y (score)
10	90
9	80
3	50
2	30

Regression

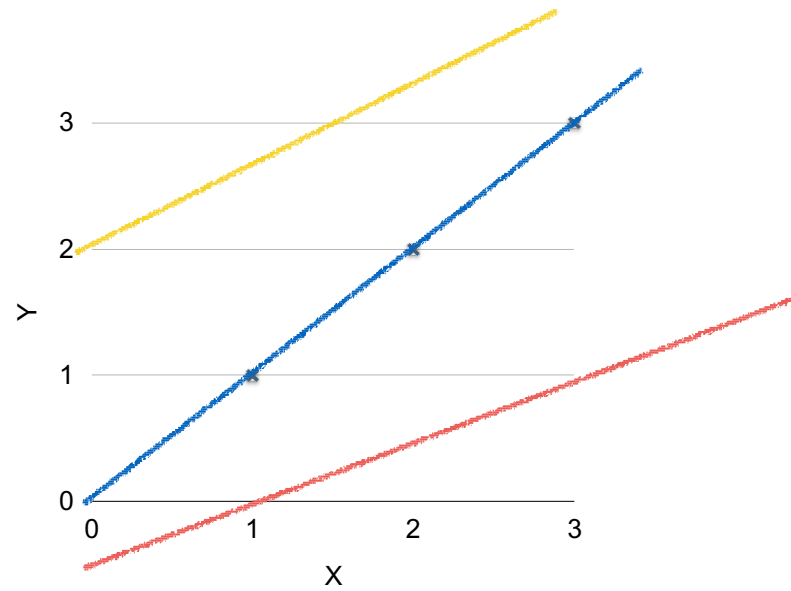
x	Y
1	1
2	2
3	3



(Linear) Hypothesis

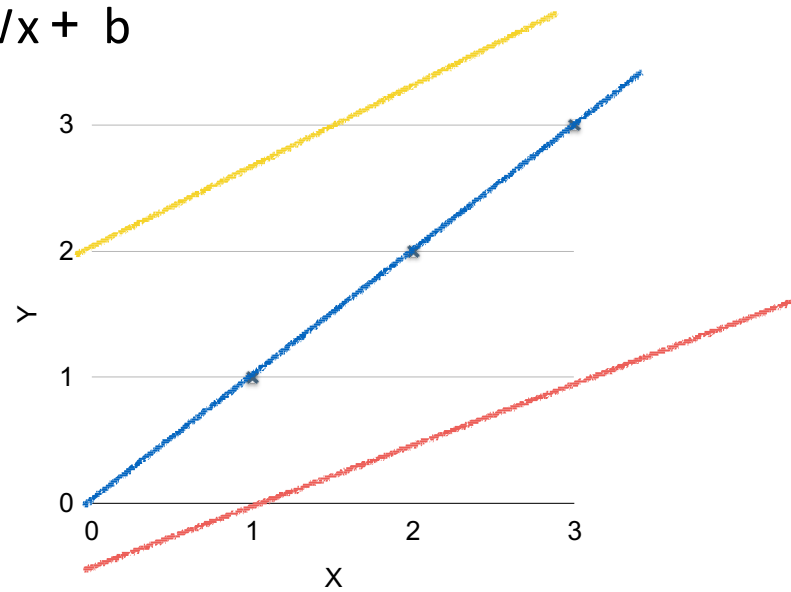


(Linear) Hypothesis

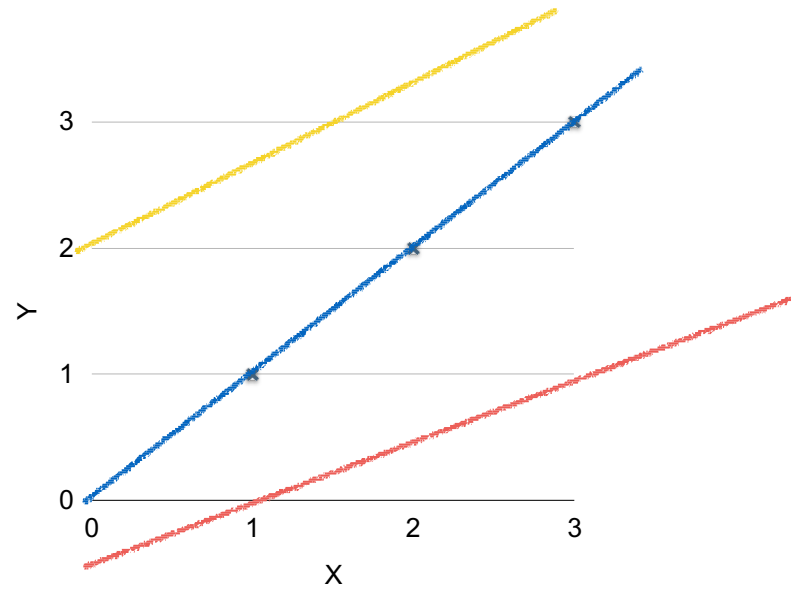


(Linear) Hypothesis

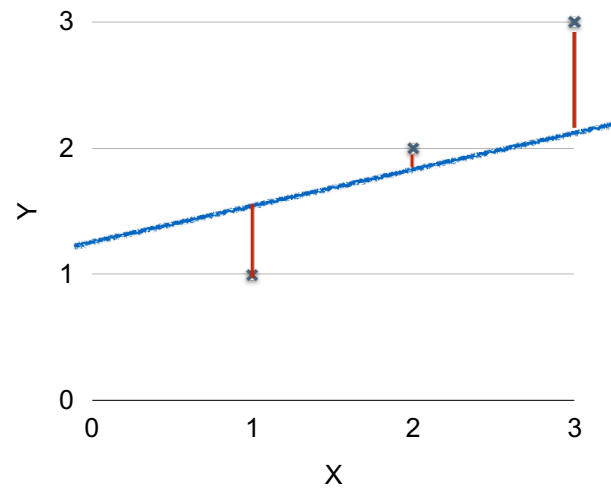
$$H(x) = Wx + b$$



Which hypothesis is better?



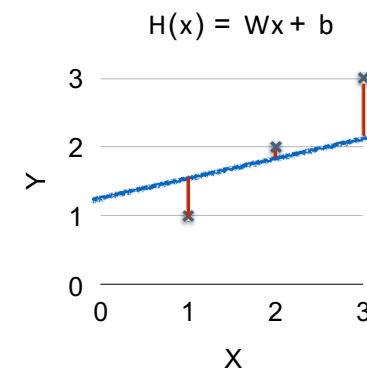
Which hypothesis is better?



Cost function

- How fit the line to our (training) data

$$H(x) \rightarrow y$$



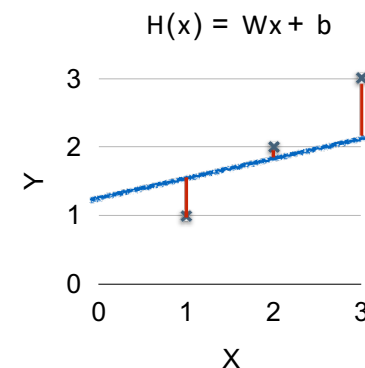
손실함수 (Loss Function)

Cost function

- How fit the line to our (training) data

$$\frac{(H(x^{(1)}) - y^{(1)})^2 + (H(x^{(2)}) - y^{(2)})^2 + (H(x^{(3)}) - y^{(3)})^2}{3}$$

$$cost = \frac{1}{m} \sum_{i=1}^m (H(x^{(i)}) - y^{(i)})^2$$



Cost function

$$cost = \frac{1}{m} \sum_{i=1}^m (H(x^{(i)}) - y^{(i)})^2$$

$$H(x) = Wx + b$$

$$cost(W, b) = \frac{1}{m} \sum_{i=1}^m (H(x^{(i)}) - y^{(i)})^2$$

Goal: Minimize cost

$$\underset{W, b}{\text{minimize}} \text{ cost}(W, b)$$

손실함수 - 오차제곱합

$$E = \frac{1}{2} \sum_k (y_k - t_k)^2$$

```
2  def sum_squares_error(y,t):
3      return 0.5*np.sum((y-t)**2)
4
5  import numpy as np
6
7  y1= [0.1, 0.05, 0.6, 0.0, 0.05, 0.1, 0.0, 0.1, 0.0, 0.01]
8  y2= [0.1, 0.05, 0.1, 0.0, 0.05, 0.1, 0.0, 0.6, 0.0, 0.01]
9  t= [0, 0, 1, 0, 0, 0, 0, 0, 0, 0]
10
11  print(sum_squares_error(np.array(y1), np.array(t)), sum_squares_error(np.array(y2), np.array(t)))
```

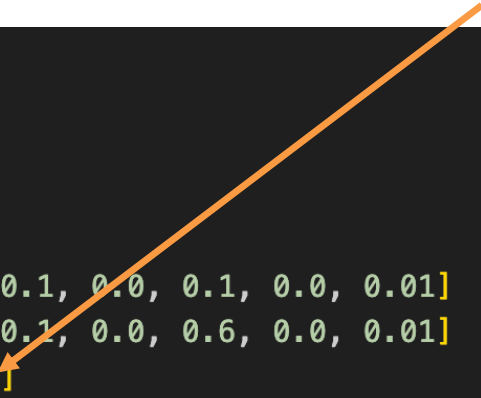
```
● limsc@Limsc-dguui-Mac-Studio Co
0.09755000000000003 0.59755
```

손실함수 - 오차제곱합

$$E = \frac{1}{2} \sum_k (y_k - t_k)^2$$

One-Hot Encoding : 0/1로 결과 표현

```
2  def sum_squares_error(y,t):
3      return 0.5*np.sum((y-t)**2)
4
5  import numpy as np
6
7  y1= [0.1, 0.05, 0.6, 0.0, 0.05, 0.1, 0.0, 0.1, 0.0, 0.01]
8  y2= [0.1, 0.05, 0.1, 0.0, 0.05, 0.1, 0.0, 0.6, 0.0, 0.01]
9  t= [0, 0, 1, 0, 0, 0, 0, 0, 0, 0]
10
11  print(sum_squares_error(np.array(y1), np.array(t)), sum_squares_error(np.array(y2), np.array(t)))
```

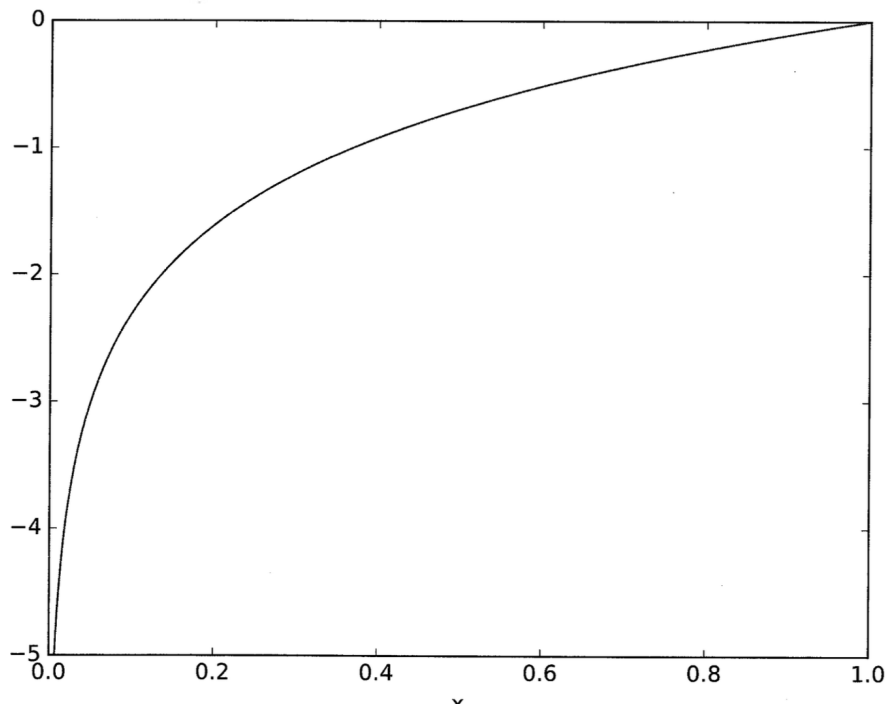


```
● limsc@Limsc-dguui-Mac-Studio Co
0.09755000000000003 0.59755
```

손실함수 - 교차엔트로피오차

$$E = -\sum_k t_k \log y_k$$

log는 밑이 e 인 자연로그($\log_e$)



- $x=1 : y=0$,
 $x \rightarrow 0, y$: decrease

손실함수 - 교차엔트로피오차

```
def cross_entropy_error(y,t):  
    delta = 1e-7  
    return -np.sum(t*np.log(y+delta))
```

Delta? : 0 입력시 $-\infty$ 이기 때문에 발생안하게

```
import numpy as np  
  
y1= [0.1, 0.05, 0.6, 0.0, 0.05, 0.1, 0.0, 0.1, 0.0, 0.01]  
y2= [0.1, 0.05, 0.1, 0.0, 0.05, 0.1, 0.0, 0.6, 0.0, 0.01]  
t= [0, 0, 1, 0, 0, 0, 0, 0, 0, 0]  
print(cross_entropy_error(np.array(y1), np.array(t)), cross_entropy_error(np.array(y2), np.array(t)))
```

```
● limsc@Limsc-dguui-Mac-Studio Codes % /usr/bin  
0.510825457099338 2.302584092994546
```

손실함수 - 미니배치학습

- 손실함수를 N 개의 데이터로 계산 후 1/N으로 정규화

$$E = -\frac{1}{N} \sum_n \sum_k t_{nk} \log y_{nk}$$

- 미니배치 : 신경망 학습에서 훈련 데이터로부터 일부만 골라서 학습
ex) mnist 60000장의 훈련데이터 중 100장을 무작위로 뽑아 100장만 사용하여 학습

$$E = -\frac{1}{N} \sum_n \sum_k t_{nk} \log y_{nk}$$

```
def cross_entropy_error(y,t): #one-hot encoding
    if y.ndim ==1:
        t=t.reshape(1,t.size)
        y=y.reshape(1,y.size)
    batch_size = y.shape[0]
    delta = 1e-7
    return -np.sum(t*np.log(y+delta))/batch_size

def cross_entropy_error2(y,t):
    if y.ndim ==1:
        t=t.reshape(1,t.size)
        y=y.reshape(1,y.size)
    batch_size = y.shape[0]
    delta = 1e-7
    return -np.sum(np.log(y[np.arange(batch_size),t]+delta))/batch_size
    #np.arange(size) : 0~size-1까지 배열 생성
```

- batch_size : 5, → np.arange(batch_size) : [0,1,2,3,4], t:[2,7,0,3,5]
 → y[np.arange(batch_size),t] : [y[0,2], y[1,7], y[2,0], y[3,9], y[4,4]]


```
import sys, os
import numpy as np
sys.path.append(os.pardir)
from dataset.mnist import load_mnist
```

```
(x_train, t_train), (x_test, t_test) = load_mnist(flatten=True, normalize=False, one_hot_label=False)
train_size = x_train.shape[0]
batch_size = 10
batch_mask = np.random.choice(train_size, batch_size)
x_batch = x_train[batch_mask]
t_batch = t_train[batch_mask]
```

```
print(x_train.shape)
print(x_test.shape)
```

```
(60000, 784)
(10000, 784)
```

```
print(train_size)
print(batch_mask)
```

```
60000
[57943 18883 40117 29447 21415 13362  5745 39724 33062 10093]
```

```
print(t_batch)
print(x_batch.ndim)
print(cross_entropy_error(t_batch, t_batch))
```

```
[6 9 7 1 3 9 9 9 7 3]
2
-123.68505841825525
```

How to minimize cost?

Hypothesis and Cost

$$H(x) = Wx + b$$

$$cost(W, b) = \frac{1}{m} \sum_{i=1}^m (H(x^{(i)}) - y^{(i)})^2$$

Simplified hypothesis

$$H(x) = Wx$$

$$\text{cost}(W) = \frac{1}{m} \sum_{i=1}^m (Wx^{(i)} - y^{(i)})^2$$

What $\text{cost}(W)$ looks like?

$$\text{cost}(W) = \frac{1}{m} \sum_{i=1}^m (W x^{(i)} - y^{(i)})^2$$

x	Y
1	1
2	2
3	3

- $W=1, \text{cost}(W)=?$

What $\text{cost}(W)$ looks like?

$$H(x) = Wx$$

$$\text{cost}(W) = \frac{1}{m} \sum_{i=1}^m (Wx^{(i)} - y^{(i)})^2$$

x	Y
1	1
2	2
3	3

- $W=1, \text{cost}(W)=0$

$$\frac{1}{3}((1 * 1 - 1)^2 + (1 * 2 - 2)^2 + (1 * 3 - 3)^2)$$

- $W=0, \text{cost}(W)=4.67$

$$\frac{1}{3}((0 * 1 - 1)^2 + (0 * 2 - 2)^2 + (0 * 3 - 3)^2)$$

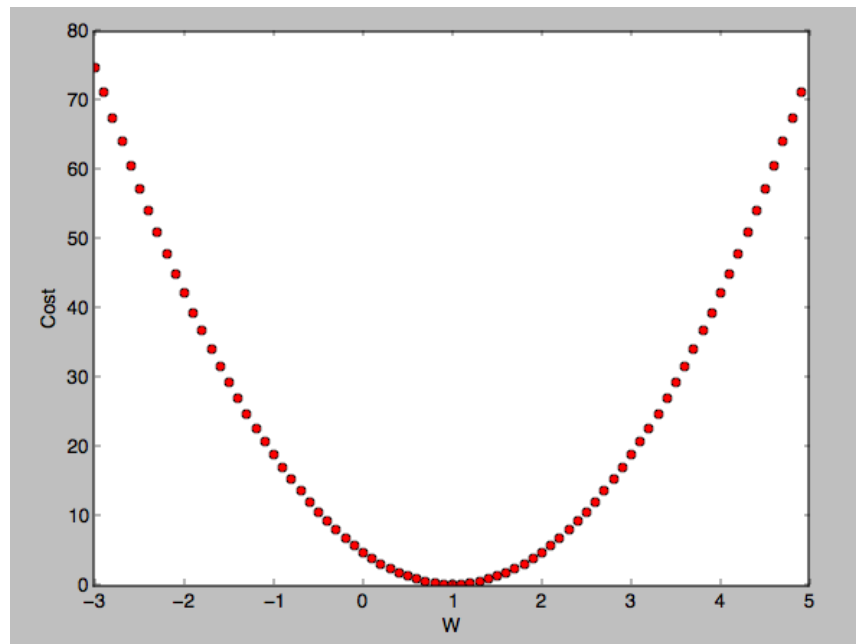
- $W=2, \text{cost}(W)=?$

What $\text{cost}(W)$ looks like?

- $W=1, \text{cost}(W)=0$
- $W=0, \text{cost}(W)=4.67$
- $W=2, \text{cost}(W)=4.67$

What $\text{cost}(W)$ looks like?

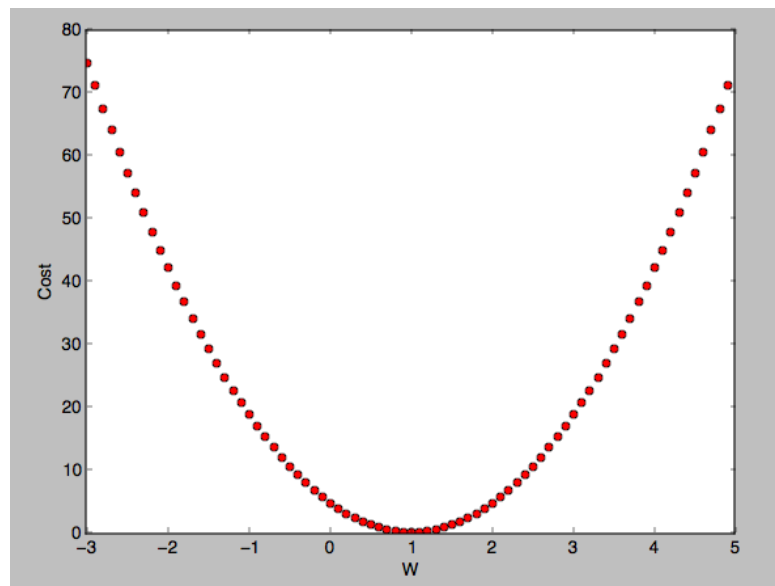
$$\text{cost}(W) = \frac{1}{m} \sum_{i=1}^m (W x^{(i)} - y^{(i)})^2$$



Gradient descent algorithm

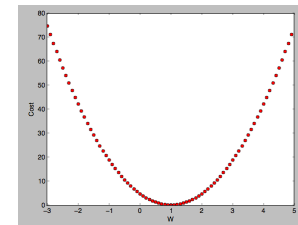
- Minimize cost function
- Gradient descent is used many minimization problems
- For a given cost function, $cost(W, b)$, it will find W, b to minimize cost
- It can be applied to more general function: $cost(w1, w2, \dots)$

How it works?
How would you find the lowest point?

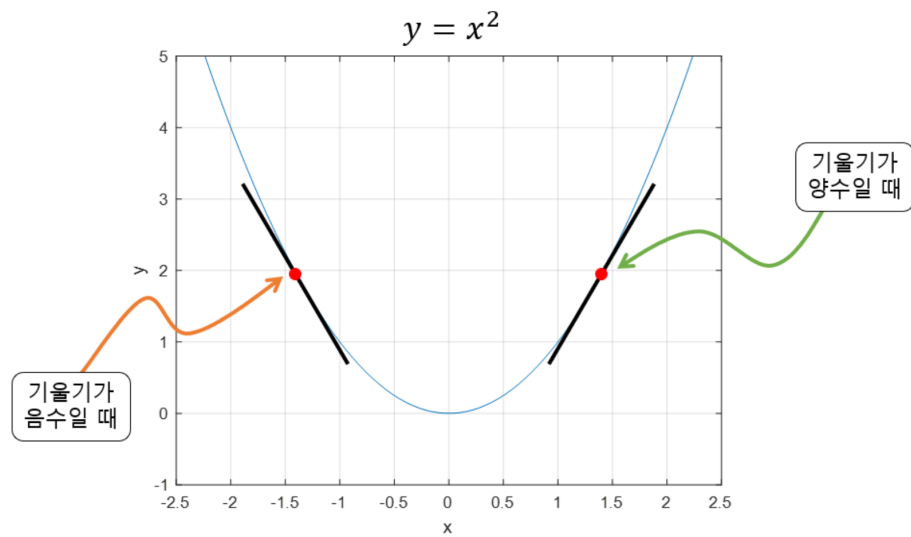


How it works?

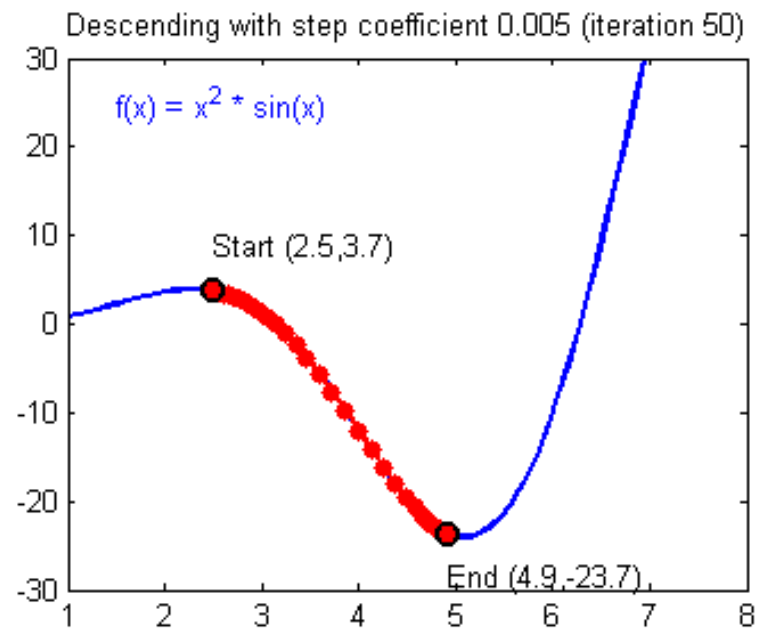
- Start with initial guesses
 - Start at 0,0 (or any other value)
 - Keeping changing W and b a little bit to try and reduce $\text{cost}(W, b)$
- Each time you change the parameters, you select the gradient which reduces $\text{cost}(W, b)$ the most possible
- Repeat
- Do so until you converge to a local minimum
- Has an interesting property
 - Where you start can determine which minimum you end up



http://www.holehouse.org/mlclass/01_02_Introduction_regression_analysis_and_gr.html

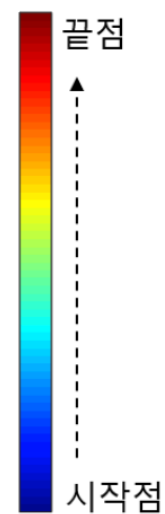
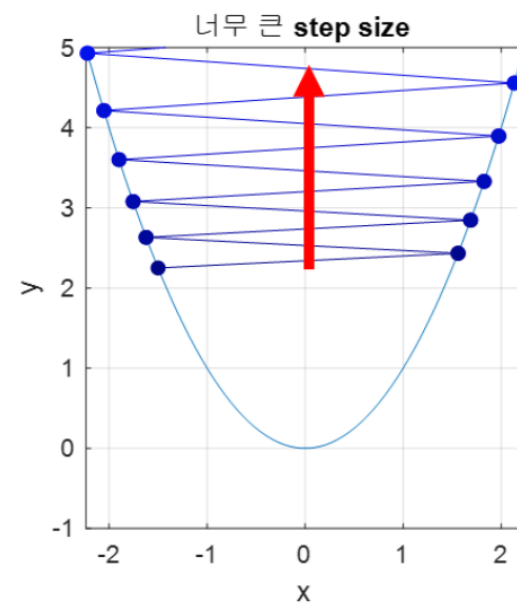
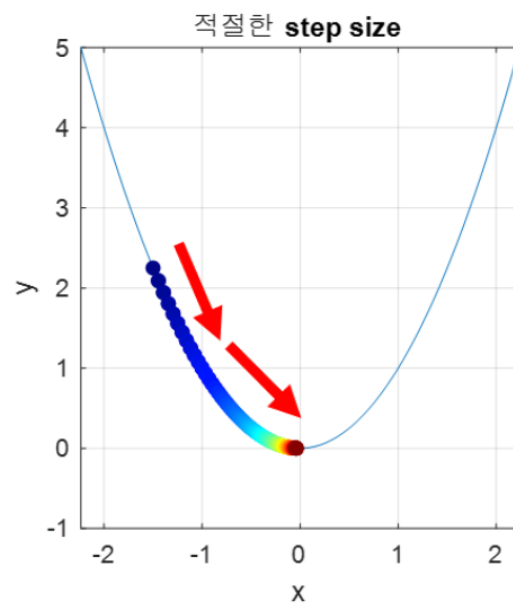
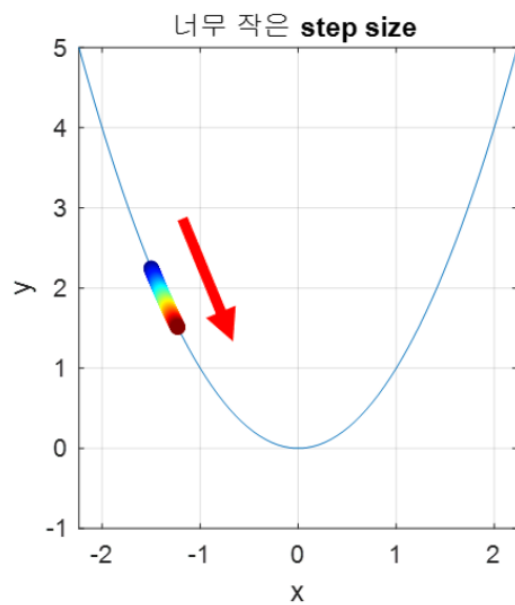


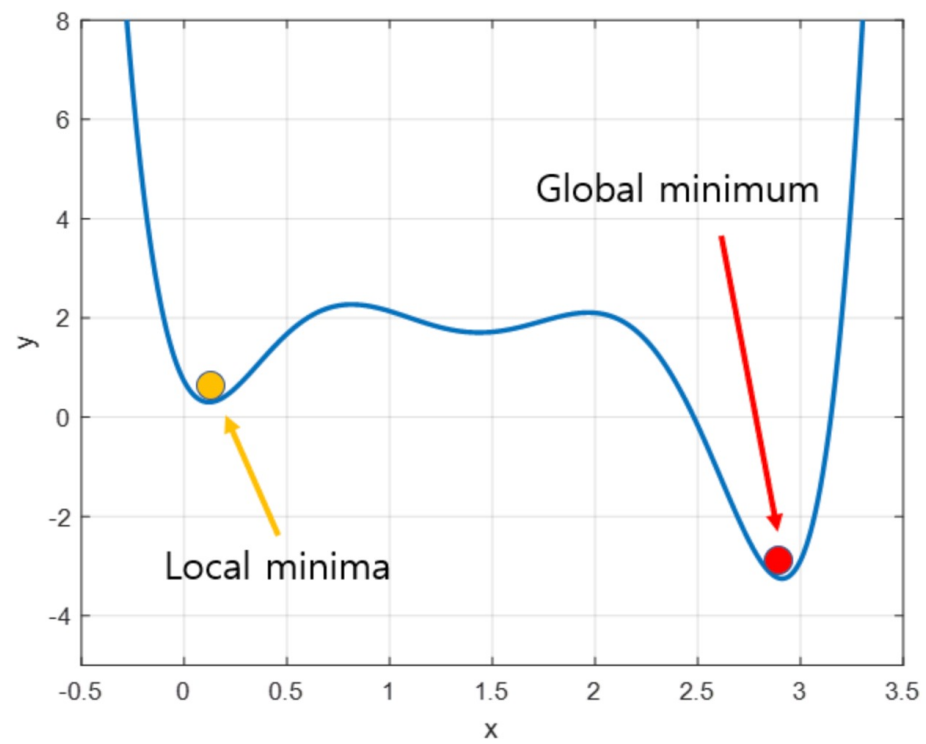
$$x_{i+1} = x_i - \text{이동 거리} \times \text{기울기의 부호}$$



$$x_{i+1} = x_i - \alpha \frac{df}{dx}(x_i)$$

$$x_{i+1} = x_i - \alpha \nabla f(x_i)$$





Formal definition

$$cost(W) = \frac{1}{m} \sum_{i=1}^m (Wx^{(i)} - y^{(i)})^2$$



$$cost(W) = \frac{1}{2m} \sum_{i=1}^m (Wx^{(i)} - y^{(i)})^2$$

Formal definition

$$cost(W) = \frac{1}{2m} \sum_{i=1}^m (W x^{(i)} - y^{(i)})^2$$

$$W := W - \alpha \frac{\partial}{\partial W} cost(W)$$

Formal definition

$$W := W - \alpha \frac{\partial}{\partial W} \frac{1}{2m} \sum_{i=1}^m (W x^{(i)} - y^{(i)})^2$$

$$W := W - \alpha \frac{1}{2m} \sum_{i=1}^m 2(W x^{(i)} - y^{(i)}) x^{(i)}$$

$$W := W - \alpha \frac{1}{m} \sum_{i=1}^m (W x^{(i)} - y^{(i)}) x^{(i)}$$

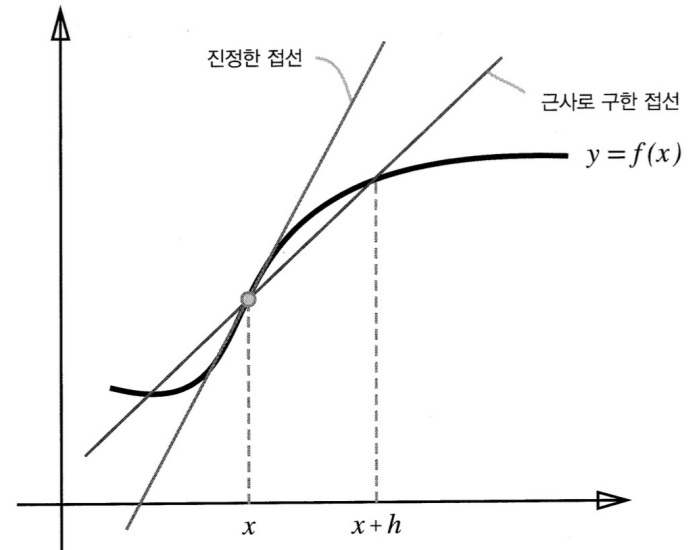
Gradient descent algorithm

$$W := W - \alpha \frac{1}{m} \sum_{i=1}^m (W x^{(i)} - y^{(i)}) x^{(i)}$$

미분

$$\frac{df(x)}{dx} = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$

```
def numerical_diff(f, x):  
    h = 1e-4 # 0.0001  
    return (f(x+h) - f(x-h)) / (2*h)
```

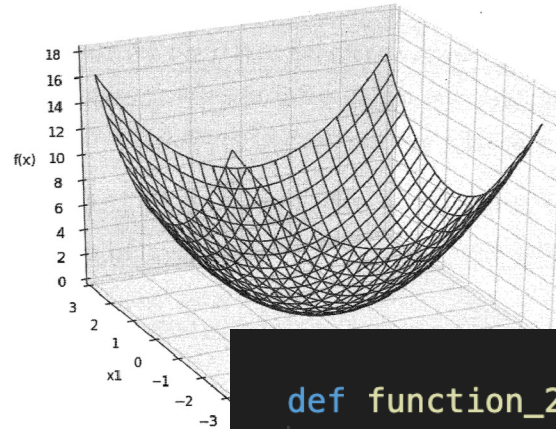


수치미분 예

$$y = 0.01x^2 + 0.1x$$

```
_week_09_diff.py > ...  
def numerical_diff(f, x):  
    h=1e-4 # 0.0001  
    return (f(x+h)-f(x-h))/(2*h)  
  
def function_1(x):  
    return 0.01*x**2 + 0.1*x  
  
import numpy as np  
import matplotlib.pyplot as plt  
  
x=np.arange(0.0,20.0,0.1)  
y=function_1(x)  
plt.plot(x,y)  
plt.show()  
  
print(numerical_diff(function_1,5))
```

편미분의 예



$$f(x_0, x_1) = x_0^2 + x_1^2$$

문제 1 : $x_0 = 3, x_1 = 4$ 일 때, x_0 에 대한 편미분 $\frac{\partial}{\partial x_0}$

```
def function_2_1(x0):      #x1=4, partial diff of x0
    return x0*x0 + 4.0*4.0
```

```
import numpy as np
import matplotlib.pyplot as plt
```

```
x=np.arange(0.0,20.0,0.1)
y=function_1(x)
plt.plot(x,y)
plt.show()
```

```
print(numerical_diff(function_1,5))
```

```
print(numerical_diff(function_2_1,3.0))
```

경사하강법

$$x_0 = x_0 - \eta \frac{\partial f}{\partial x_0}$$

$$x_1 = x_1 - \eta \frac{\partial f}{\partial x_1}$$

- η : learning rate : 한번의 학습으로 얼마만큼 학습해야할지. 매개변수 값을 얼마나 갱신할지

```
def _numerical_gradient_no_batch(f, x):
    h = 1e-4 # 0.0001
    grad = np.zeros_like(x) # x와 형상이 같은 배열을 생성

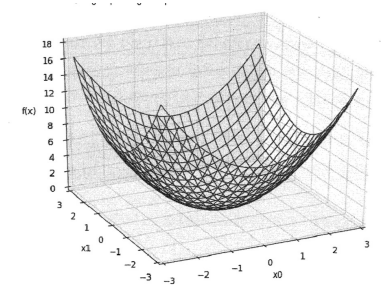
    for idx in range(x.size):
        tmp_val = x[idx]

        # f(x+h) 계산
        x[idx] = float(tmp_val) + h
        fxh1 = f(x)

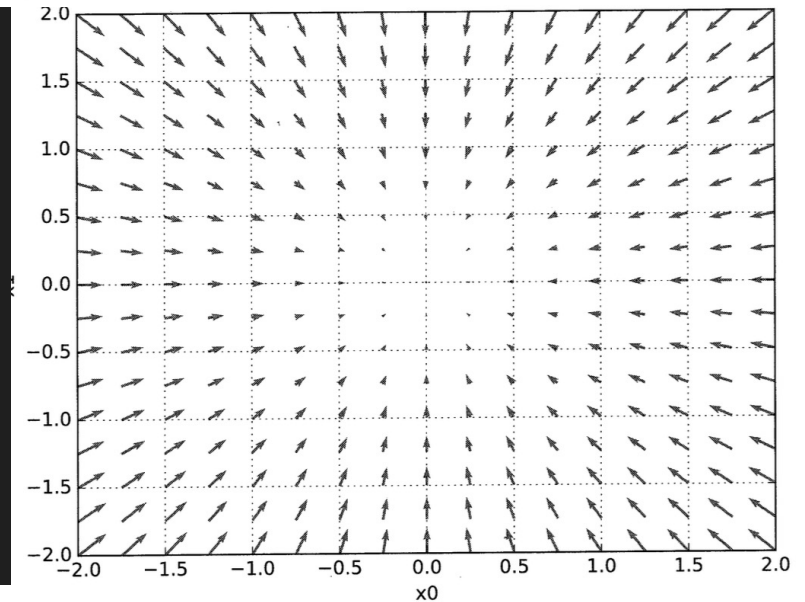
        # f(x-h) 계산
        x[idx] = tmp_val - h
        fxh2 = f(x)

        grad[idx] = (fxh1 - fxh2) / (2*h)
        x[idx] = tmp_val # 값 복원

    return grad
```



$f(x_0, x_1) = x_0^2 + x_1^2$ 의 기울기



```
print(_numerical_gradient_no_batch(function_2, np.array([3., 4.])))
```

[6. 8.]


```
def numerical_gradient(f, X):
    if X.ndim == 1:
        return _numerical_gradient_no_batch(f, X)
    else:
        grad = np.zeros_like(X)

        for idx, x in enumerate(X):
            grad[idx] = _numerical_gradient_no_batch(f, x)

        return grad
```

```
def function_2(x):
    if x.ndim == 1:
        return np.sum(x**2)
    else:
        return np.sum(x**2, axis=1)
```

$$x_0 = x_0 - \eta \frac{\partial f}{\partial x_0}$$

$$x_1 = x_1 - \eta \frac{\partial f}{\partial x_1}$$

```
def gradient_descent(f, init_x, lr=0.01, step_num=100):
    x = init_x
    x_history = []

    for i in range(step_num):
        x_history.append( x.copy() )

        grad = numerical_gradient(f, x)
        x -= lr * grad

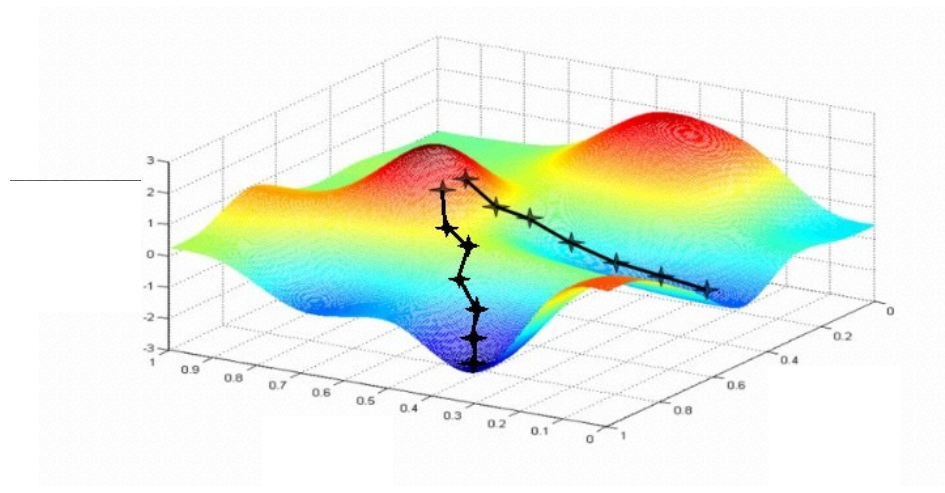
    return x#, np.array(x_history)
```

신경망에서 기울기

$$\mathbf{W} = \begin{pmatrix} w_{11} & w_{21} & w_{31} \\ w_{12} & w_{22} & w_{32} \end{pmatrix}$$

$$\frac{\partial L}{\partial \mathbf{W}} = \begin{pmatrix} \frac{\partial L}{\partial w_{11}} & \frac{\partial L}{\partial w_{21}} & \frac{\partial L}{\partial w_{31}} \\ \frac{\partial L}{\partial w_{12}} & \frac{\partial L}{\partial w_{22}} & \frac{\partial L}{\partial w_{32}} \end{pmatrix}$$

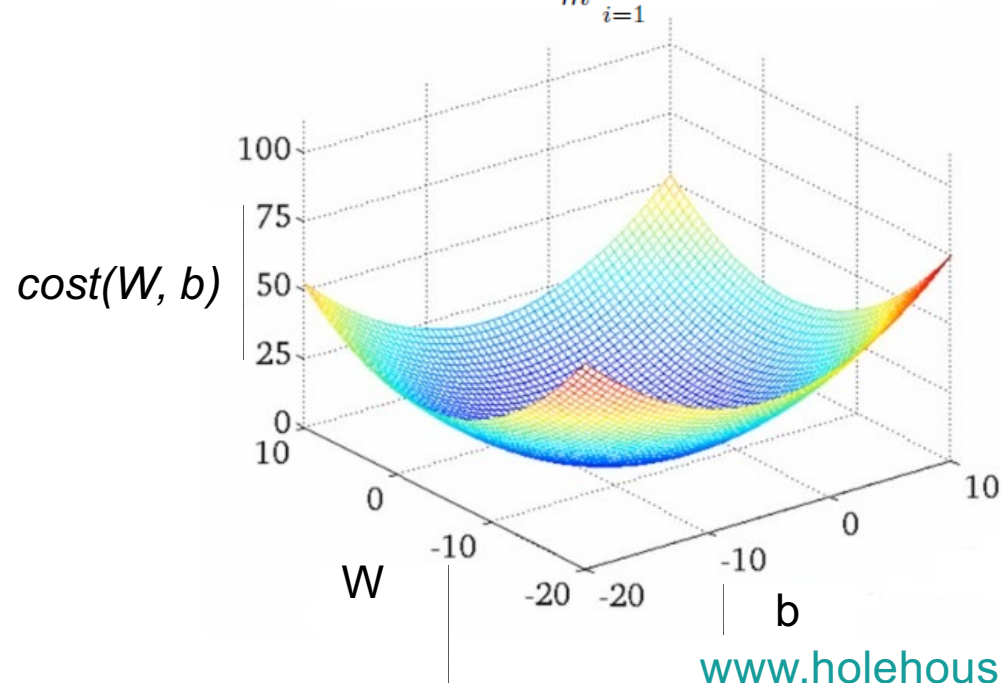
Convex function



www.holehouse.org/mlclass/

Convex function

$$cost(W, b) = \frac{1}{m} \sum_{i=1}^m (H(x^{(i)}) - y^{(i)})^2$$



www.holehouse.org/mlclass/

오늘 할 일

$$f(x_0, x_1) = x_0^2 + x_1^2$$

1. 임의의 x_0, x_1 값에 대하여 f 의 최솟값을 gradient descent를 이용하여 구하라.

2. Mnist dataset을 활용하여 100의 batch 크기로

Sample_weight이 있는 것을 활용하여 3layer NN에 적용하였을때 accuracy를 구하는 코드를 작성하시오

